

02_feature_table_check

July 15, 2026

1 02 — Phase 1: Occupation feature table + melted training rows

Purpose: run `build_occupation_feature_table()` and `build_training_rows()` against the real O*NET + BLS data, sanity-check the output shapes and values, then resolve any nulls found before writing the actual model-training script.

```
[1]: import sys
      from pathlib import Path

      project_root = Path.cwd().parent
      sys.path.insert(0, str(project_root))

      import pandas as pd

      from src.data.onet_loader import (
          load_occupation_data,
          load_essential_skills,
          load_transferable_skills,
          load_job_zones,
          build_full_skill_importance_matrix,
      )
      from src.data.oews_loader import load_national_wages
      from src.features.occupation_features import (
          build_occupation_feature_table,
          build_training_rows,
          clean_training_rows,
      )
```

```
[2]: occ = load_occupation_data()
      skills = load_essential_skills()
      transferable = load_transferable_skills()
      job_zones = load_job_zones()
      wages = load_national_wages()

      skill_matrix = build_full_skill_importance_matrix(skills, transferable)

      print(f"Occupations: {len(occ)}")
      print(f"Skill matrix: {skill_matrix.shape}")
```

```
print(f"Job zones: {len(job_zones)}")
print(f"Job zone value counts:\n{job_zones['job_zone'].value_counts().
↪sort_index()}")
```

[oews_loader] 7 rows have suppressed wage data - kept as NaN, not dropped.

Occupations: 1016

Skill matrix: (894, 35)

Job zones: 923

Job zone value counts:

job_zone

2 331

3 212

4 226

5 154

Name: count, dtype: int64

```
[3]: feature_table = build_occupation_feature_table(occ, skill_matrix, job_zones, u
↪wages)
print(f"Feature table shape: {feature_table.shape}")
print(f"Columns: {list(feature_table.columns)}")
print(f"\nOccupations with a wage value: {feature_table['a_median'].notna().
↪sum()}")
print(f"Occupations missing job_zone: {feature_table['job_zone'].isna().sum()}")
print(f"Occupations missing ALL skill values: {feature_table[skill_matrix.
↪columns].isna().all(axis=1).sum()}")
feature_table.head()
```

Feature table shape: (1016, 43)

Columns: ['title', 'job_zone', 'Active Learning', 'Active Listening', 'Complex Problem Solving', 'Coordination', 'Critical Thinking', 'Equipment Maintenance', 'Equipment Selection', 'Installation', 'Instructing', 'Judgment and Decision Making', 'Learning Strategies', 'Management of Financial Resources', 'Management of Material Resources', 'Management of Personnel Resources', 'Mathematics', 'Monitoring', 'Negotiation', 'Operation and Control', 'Operations Analysis', 'Operations Monitoring', 'Persuasion', 'Programming', 'Quality Control Analysis', 'Reading Comprehension', 'Repairing', 'Science', 'Service Orientation', 'Social Perceptiveness', 'Speaking', 'Systems Analysis', 'Systems Evaluation', 'Technology Design', 'Time Management', 'Troubleshooting', 'Writing', 'tot_emp', 'a_pct10', 'a_pct25', 'a_pct75', 'a_pct90']

Occupations with a wage value: 956

Occupations missing job_zone: 93

Occupations missing ALL skill values: 122

```
[3]:
```

onetsoc_code	title	job_zone	Active Learning	\
11-1011.00	Chief Executives	5.0	3.75	
11-1011.03	Chief Sustainability Officers	5.0	3.75	

11-1021.00	General and Operations Managers	4.0	3.62
11-1031.00	Legislators	4.0	NaN
11-2011.00	Advertising and Promotions Managers	4.0	3.25

	Active Listening	Complex Problem Solving	Coordination \
onetsoc_code			
11-1011.00	4.00	4.38	4.25
11-1011.03	4.00	4.00	3.75
11-1021.00	4.00	3.62	3.88
11-1031.00	NaN	NaN	NaN
11-2011.00	4.12	3.50	3.50

	Critical Thinking	Equipment Maintenance	Equipment Selection \
onetsoc_code			
11-1011.00	4.38	1.0	1.12
11-1011.03	4.12	1.0	1.12
11-1021.00	3.88	1.0	1.00
11-1031.00	NaN	NaN	NaN
11-2011.00	4.00	1.0	1.12

	Installation ...	Technology Design	Time Management \
onetsoc_code	...		
11-1011.00	1.0 ...	1.75	4.00
11-1011.03	1.0 ...	1.88	3.38
11-1021.00	1.0 ...	1.50	3.62
11-1031.00	NaN ...	NaN	NaN
11-2011.00	1.0 ...	1.75	3.50

	Troubleshooting	Writing	tot_emp	a_pct10	a_pct25 \
onetsoc_code					
11-1011.00	1.50	4.12	204350.0	75700.0	129540.0
11-1011.03	1.00	4.12	204350.0	75700.0	129540.0
11-1021.00	1.75	3.50	3503020.0	50090.0	72320.0
11-1031.00	NaN	NaN	NaN	NaN	NaN
11-2011.00	1.00	3.75	21470.0	63300.0	91370.0

	a_median	a_pct75	a_pct90
onetsoc_code			
11-1011.00	213990.0	356200.0	507730.0
11-1011.03	213990.0	356200.0	507730.0
11-1021.00	105770.0	167280.0	253390.0
11-1031.00	NaN	NaN	NaN
11-2011.00	133660.0	201050.0	286240.0

[5 rows x 43 columns]

```
[4]: training_rows = build_training_rows(feature_table, skill_cols=list(skill_matrix.
    ↪columns))
print(f"Training rows shape: {training_rows.shape}")
print(f"Unique occupations contributing rows: {training_rows['onetsoc_code'].
    ↪nunique()}")
print(f"Expected: unique occupations x 5 = {training_rows['onetsoc_code'].
    ↪nunique() * 5}")
print(f"\nAny nulls remaining in training rows?\n{training_rows.isna().sum().
    ↪sum()} total null values")
training_rows.head(10)
```

Training rows shape: (4780, 39)

Unique occupations contributing rows: 956

Expected: unique occupations x 5 = 4780

Any nulls remaining in training rows?

16795 total null values

```
[4]:  onetsoc_code  job_zone  Active Learning  Active Listening  \
0    11-1011.00      5.0           3.75           4.00
1    11-1011.03      5.0           3.75           4.00
2    11-1021.00      4.0           3.62           4.00
3    11-2011.00      4.0           3.25           4.12
4    11-2021.00      4.0           3.88           3.88
5    11-2022.00      4.0           3.75           4.00
6    11-2032.00      4.0           NaN           NaN
7    11-2033.00      4.0           3.38           4.00
8    11-3012.00      3.0           3.12           4.00
9    11-3013.00      3.0           3.12           3.75

      Complex Problem Solving  Coordination  Critical Thinking  \
0                4.38           4.25           4.38
1                4.00           3.75           4.12
2                3.62           3.88           3.88
3                3.50           3.50           4.00
4                3.62           3.50           3.88
5                3.75           3.75           3.88
6                NaN           NaN           NaN
7                3.62           3.75           4.12
8                3.25           3.75           3.75
9                3.38           3.62           3.75

      Equipment Maintenance  Equipment Selection  Installation  ...  \
0                1.00           1.12           1.00  ...
1                1.00           1.12           1.00  ...
2                1.00           1.00           1.00  ...
3                1.00           1.12           1.00  ...
```

4	1.00	1.00	1.00	...
5	1.00	1.00	1.00	...
6	NaN	NaN	NaN	...
7	1.00	1.00	1.00	...
8	1.00	1.12	1.00	...
9	2.38	2.25	1.12	...

	Social Perceptiveness	Speaking	Systems Analysis	Systems Evaluation	\
0	4.12	4.25	4.12	4.25	
1	3.88	4.00	3.88	3.88	
2	3.75	4.00	3.12	3.12	
3	4.00	4.00	3.12	3.12	
4	3.88	3.88	3.25	3.50	
5	3.88	4.00	3.62	3.62	
6	NaN	NaN	NaN	NaN	
7	4.00	4.12	3.12	3.00	
8	3.25	3.88	3.12	2.75	
9	3.62	3.88	3.00	3.00	

	Technology Design	Time Management	Troubleshooting	Writing	percentile	\
0	1.75	4.00	1.50	4.12	10	
1	1.88	3.38	1.00	4.12	10	
2	1.50	3.62	1.75	3.50	10	
3	1.75	3.50	1.00	3.75	10	
4	1.75	3.50	1.00	3.25	10	
5	1.50	3.75	1.00	3.62	10	
6	NaN	NaN	NaN	NaN	10	
7	1.75	3.50	1.00	3.88	10	
8	1.50	4.00	1.88	3.75	10	
9	1.62	3.38	2.62	3.25	10	

	log_wage
0	11.234533
1	11.234533
2	10.821577
3	11.055641
4	11.410450
5	11.200541
6	11.319462
7	11.226975
8	11.113939
9	11.074110

[10 rows x 39 columns]

1.1 Sanity checks before moving to model training

1. Training row count should be (# occupations with at least one wage value) x 5.

2. No nulls should remain in the training rows (job_zone or skill columns missing would break model training silently otherwise).
3. log_wage should be strictly increasing with percentile within each occupation — confirms the melt paired percentiles/wages correctly.

```
[5]: # Check 2: any nulls in job_zone or skill columns for the rows we're about to
      ↪train on?
null_counts = training_rows.isna().sum()
null_counts = null_counts[null_counts > 0]
if len(null_counts) > 0:
    print("WARNING - nulls found in training rows:")
    print(null_counts)
    print("\nThese need to be resolved (imputation or row removal) before model_
    ↪training.")
else:
    print("No nulls in training rows - clean to proceed to model training.")
```

WARNING - nulls found in training rows:

job_zone	345
Active Learning	470
Active Listening	470
Complex Problem Solving	470
Coordination	470
Critical Thinking	470
Equipment Maintenance	470
Equipment Selection	470
Installation	470
Instructing	470
Judgment and Decision Making	470
Learning Strategies	470
Management of Financial Resources	470
Management of Material Resources	470
Management of Personnel Resources	470
Mathematics	470
Monitoring	470
Negotiation	470
Operation and Control	470
Operations Analysis	470
Operations Monitoring	470
Persuasion	470
Programming	470
Quality Control Analysis	470
Reading Comprehension	470
Repairing	470
Science	470
Service Orientation	470
Social Perceptiveness	470
Speaking	470

Systems Analysis	470
Systems Evaluation	470
Technology Design	470
Time Management	470
Troubleshooting	470
Writing	470
dtype: int64	

These need to be resolved (imputation or row removal) before model training.

```
[6]: # Check 3: log_wage strictly increasing with percentile, per occupation
violations = 0
for code, group in training_rows.groupby("onetsoc_code"):
    sorted_group = group.sort_values("percentile")
    if not sorted_group["log_wage"].is_monotonic_increasing:
        violations += 1

print(f"Occupations with non-monotonic wage percentiles: {violations} / {
    ↪{training_rows['onetsoc_code'].nunique()}}")
if violations == 0:
    print("All clean - every occupation's percentiles increase monotonically as
    ↪expected.")
```

Occupations with non-monotonic wage percentiles: 0 / 956

All clean - every occupation's percentiles increase monotonically as expected.

1.2 Resolving the nulls Check 2 found

Two distinct, separate null sources are expected here — confirmed against this real data:

1. **Occupations missing ALL skill values** (no O*NET skill survey data at all — the same ~122-occupation gap from the Phase 0 EDA, here overlapping with the wage-matched set). These rows have zero signal to train on and are **dropped**. They are NOT the same as the 60 crosswalk-unmatched occupations from Phase 0 — these have wages, just no skill data. They'll need separate handling at inference time later.
2. **Occupations with skill data but missing job_zone only** (a smaller, separate gap). job_zone is **imputed with the dataset median** — a simple, defensible choice given it's a coarse 1-5 ordinal signal, but a real modeling choice worth stating honestly in the README's limitations section rather than treating as neutral.

clean_training_rows() handles both, and asserts zero nulls remain afterward — if a third, unexpected null source ever appears, this will raise loudly rather than silently pass bad data into training.

```
[7]: cleaned_training_rows = clean_training_rows(training_rows,
    ↪skill_cols=list(skill_matrix.columns))

print(f"\nCleaned training rows shape: {cleaned_training_rows.shape}")
```

```
print(f"Unique occupations remaining: {cleaned_training_rows['onetsoc_code'].
↪nunique()}")
print(f"Total nulls remaining: {cleaned_training_rows.isna().sum().sum()}␣
↪(should be 0)")
```

Dropped 470 rows (94 occupations) missing all skill values, out of 4780 total rows.

No job_zone imputation needed.

Cleaned training rows shape: (4310, 39)

Unique occupations remaining: 862

Total nulls remaining: 0 (should be 0)

[8]: *# Re-run Check 3 on the CLEANED data, to confirm cleaning didn't disturb the
percentile ordering within any surviving occupation.*

```
violations = 0
for code, group in cleaned_training_rows.groupby("onetsoc_code"):
    sorted_group = group.sort_values("percentile")
    if not sorted_group["log_wage"].is_monotonic_increasing:
        violations += 1

print(
    f"Occupations with non-monotonic wage percentiles (post-cleaning): "
    f"{violations} / {cleaned_training_rows['onetsoc_code'].nunique()}"
)
if violations == 0:
    print("All clean - ready to move to model training (Step 6).")
```

Occupations with non-monotonic wage percentiles (post-cleaning): 0 / 862

All clean - ready to move to model training (Step 6).